

1. Install VirtualBox/VMware Workstation with different flavors of linux or windows OS on top of windows7 or 8.

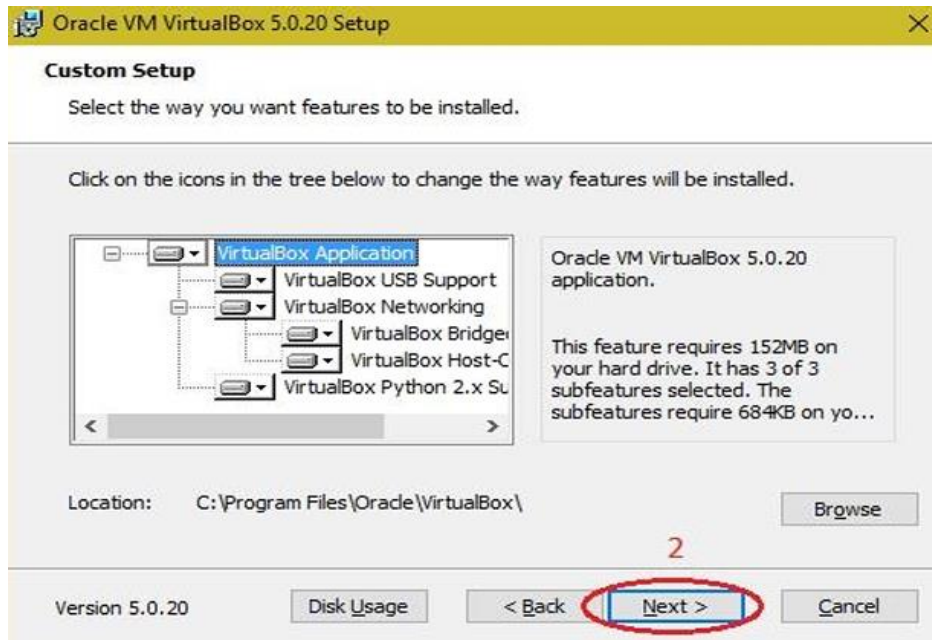
Aim: To Install Virtualbox / VMware Workstation with different flavours of linux or windows OS.

PROCEDURE:

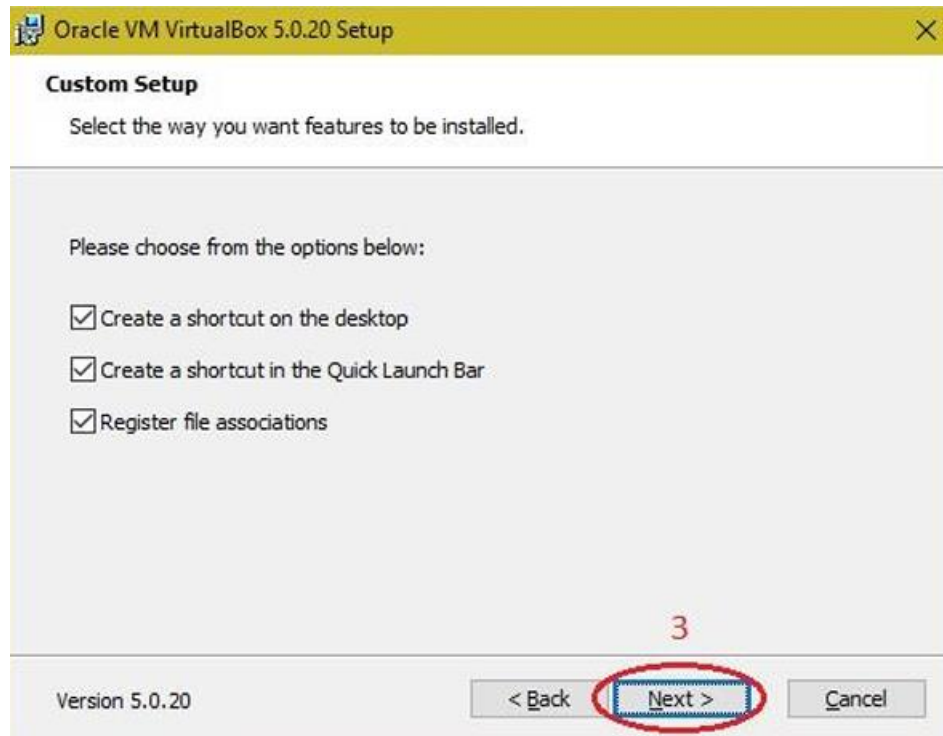
1. Download the Virtual box exe and click the exe file...and select next button.



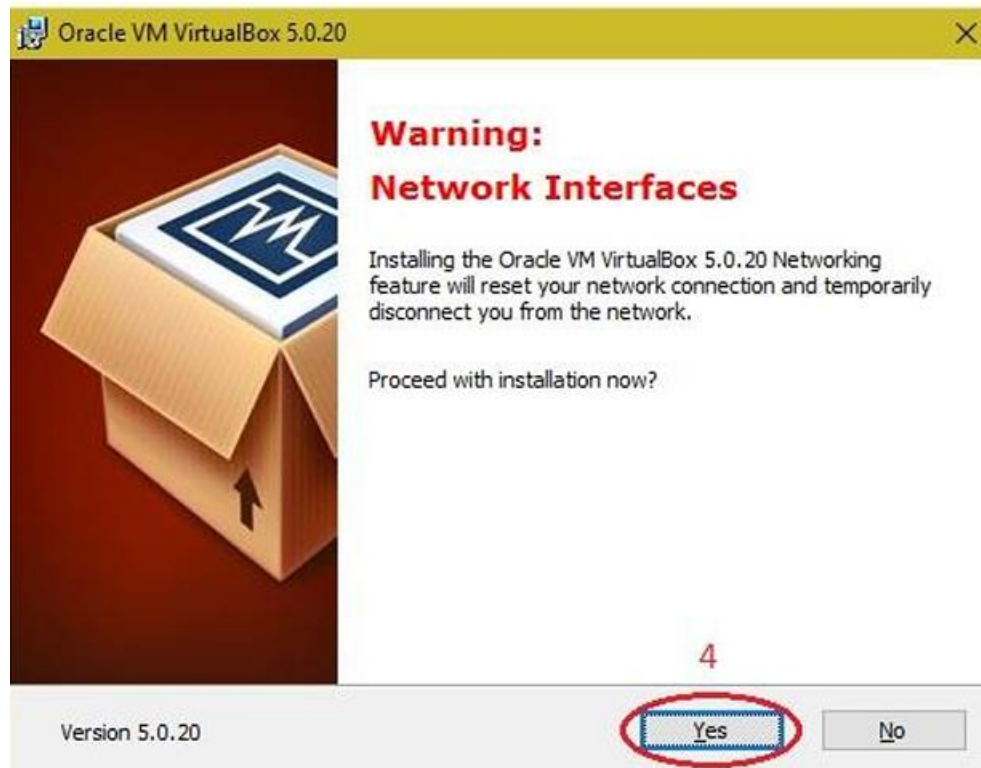
2. Click the next button.



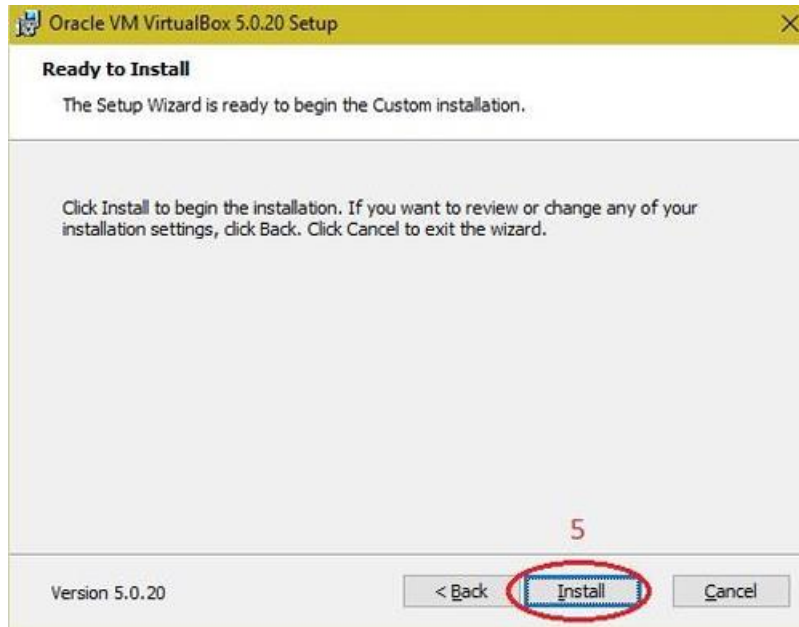
3. Click the next button.



4. Click the YES button.



5. Click the install button...



6. Then installation was completed. The show virtual box icon on desktop screen....



VirtualBox

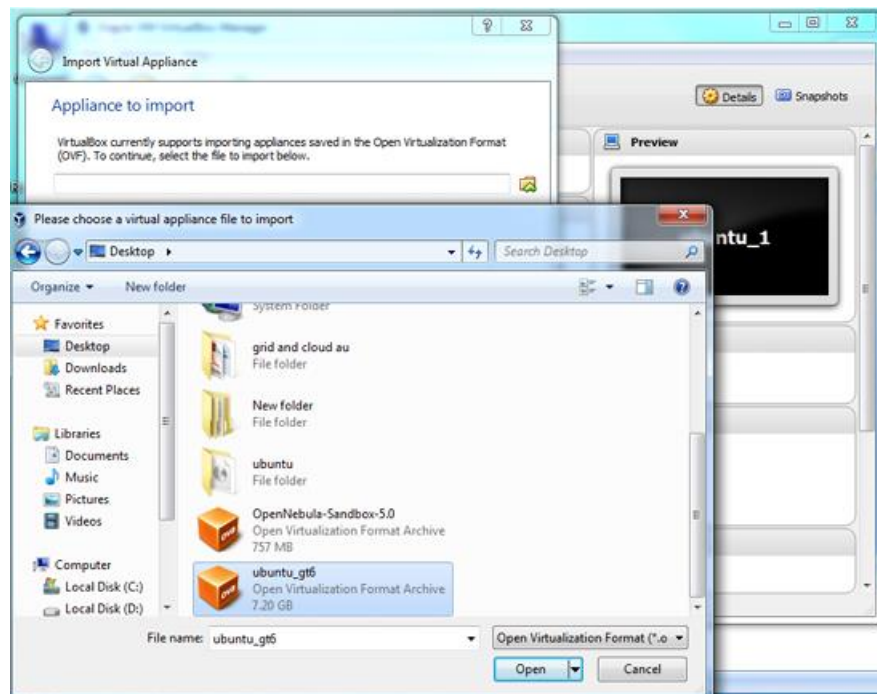
2. Install a C compiler in the virtual machine created using virtual box and executes Simple program.

Aim: To Install a C compiler in the virtual machine created using virtual box and executes Simple Program.

PROCEDURE:

Steps to import Ubuntu file

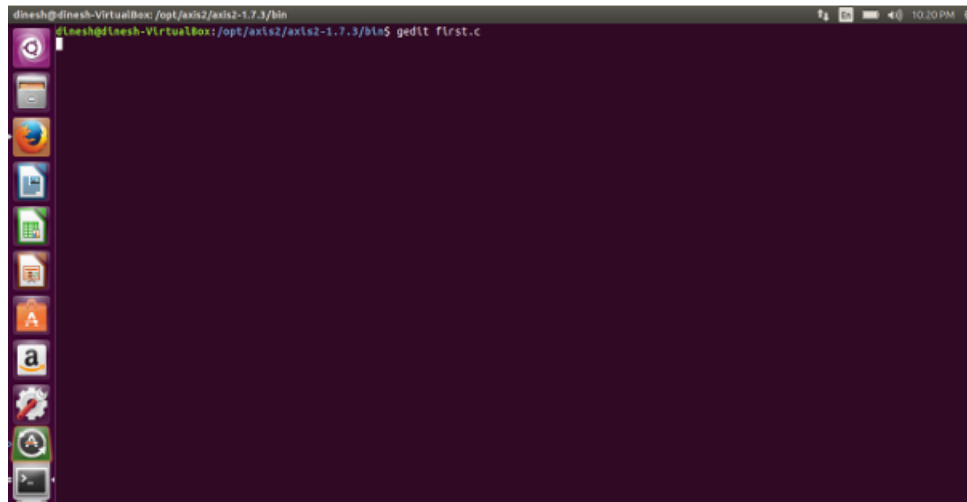
1. Open Virtual box
2. File > Import Appliance
3. Browse ubuntu_gt6.ova file
4. Then go to setting, select USB and choose USB 1.1
5. Then Start the ubuntu_gt6



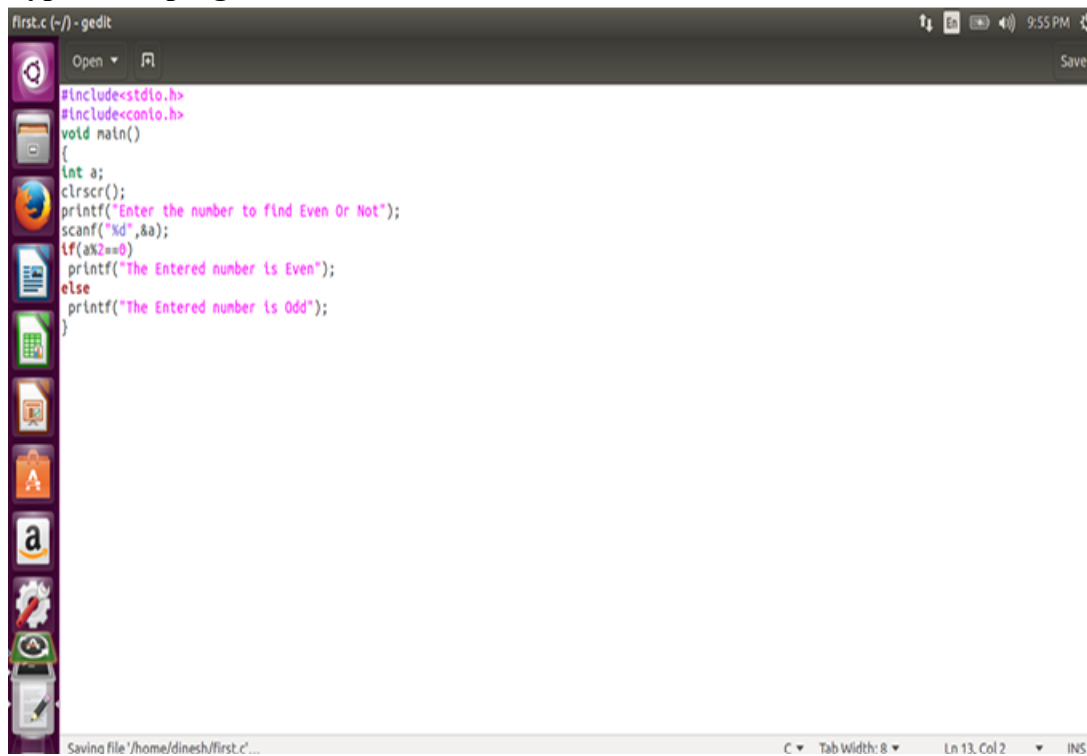
Steps to run C program:

1. Open the terminal
2. Type `cd /opt/axis2/axis2-1.7.3/bin` then press enter
3. `gedit hello.c`
4. `gcc hello.c`
5. `./a.out`

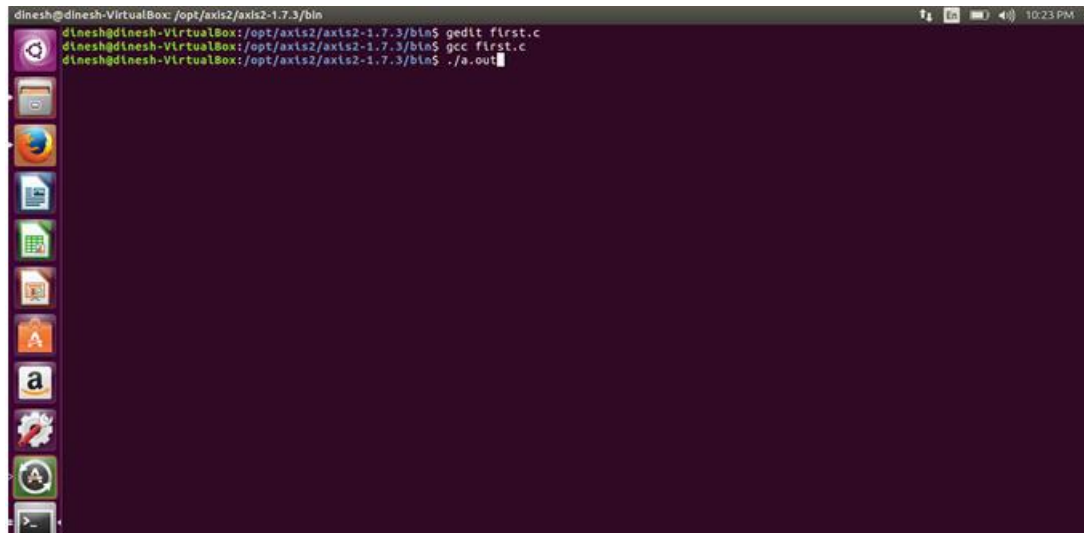
Type gedit first.c



Type the C program

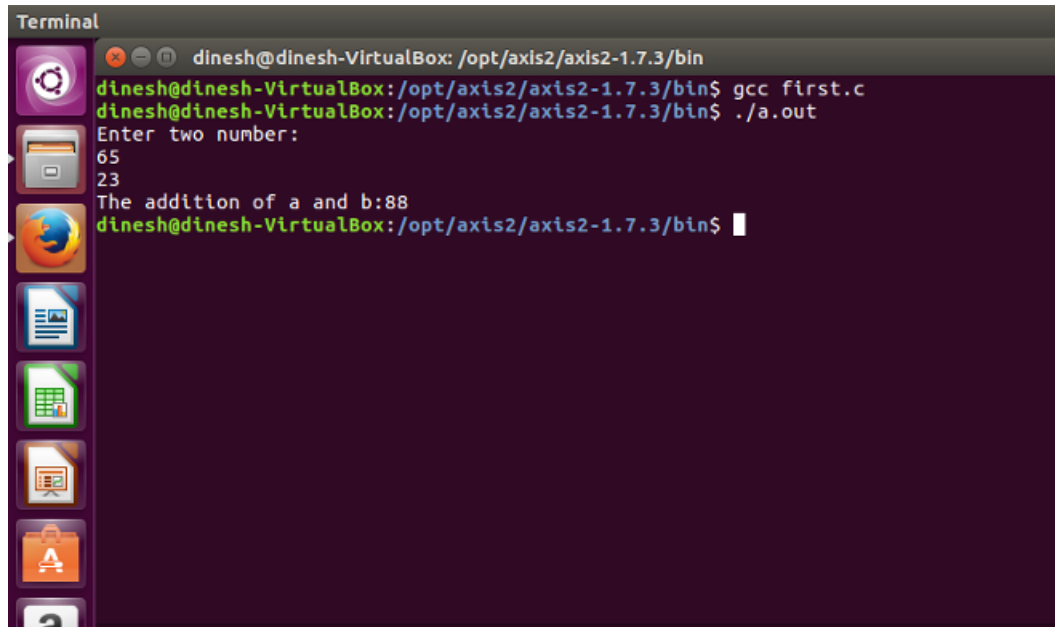


Running the C program



```
dinesh@dinesh-VirtualBox: /opt/axis2/axis2-1.7.3/bin
dinesh@dinesh-VirtualBox: /opt/axis2/axis2-1.7.3/bin$ gedit first.c
dinesh@dinesh-VirtualBox: /opt/axis2/axis2-1.7.3/bin$ gcc first.c
dinesh@dinesh-VirtualBox: /opt/axis2/axis2-1.7.3/bin$ ./a.out
```

Display the output:



```
Terminal
dinesh@dinesh-VirtualBox: /opt/axis2/axis2-1.7.3/bin
dinesh@dinesh-VirtualBox: /opt/axis2/axis2-1.7.3/bin$ gcc first.c
dinesh@dinesh-VirtualBox: /opt/axis2/axis2-1.7.3/bin$ ./a.out
Enter two number:
65
23
The addition of a and b:88
dinesh@dinesh-VirtualBox: /opt/axis2/axis2-1.7.3/bin$
```

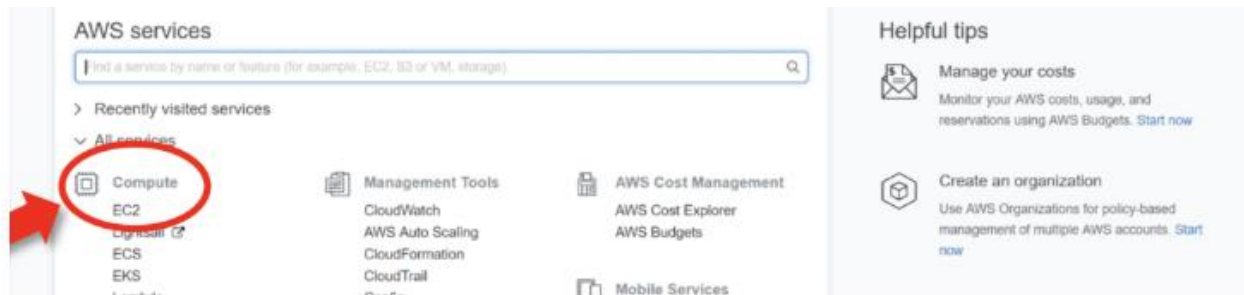
3. Create EC2 Instance in AWS (Amazon)

Aim: Create EC2 Instance in AWS (Amazon)

PROCEDURE:

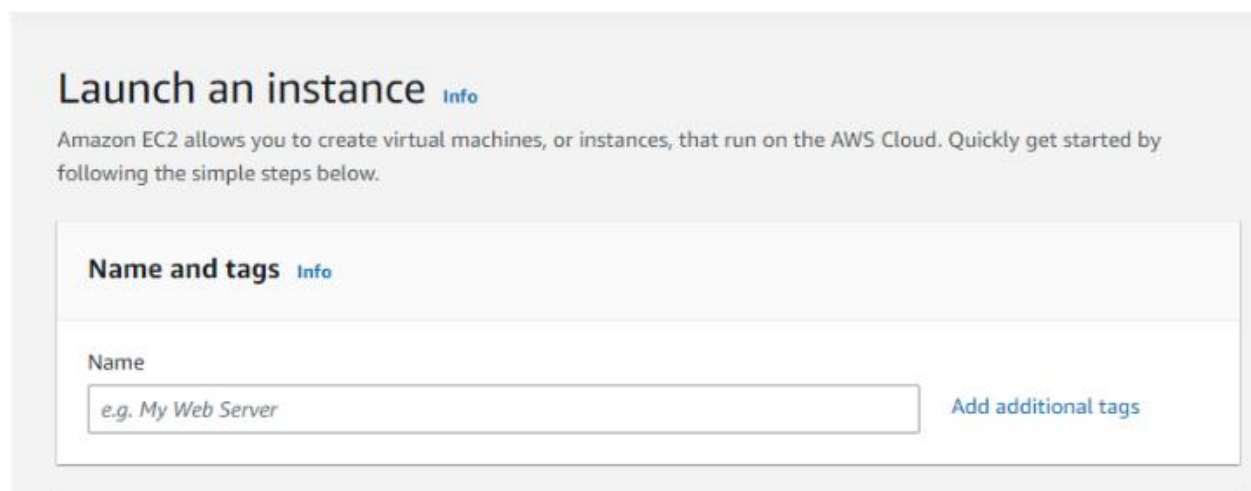
The following are the steps for creating an EC2 instance in AWS (Amazon):

Step 1: First, log into your [AWS account](#) and click on “services” present on the left of the AWS management console, i.e. the primary screen. From the drop-down menu of options, tap on “EC2”. To create an AWS free tier account refer to [Amazon Web Services \(AWS\) – Free Tier Account Set up](#).

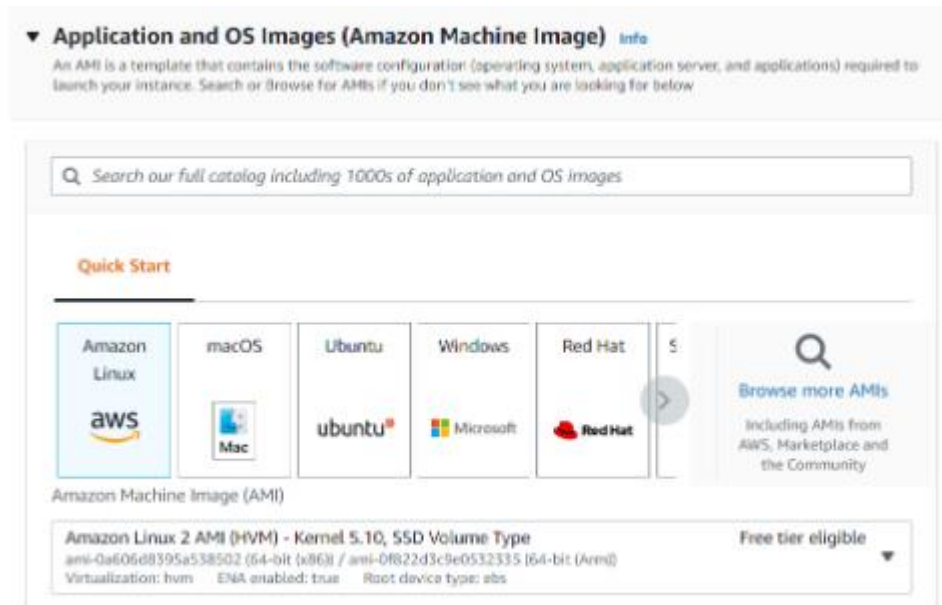


Under Resources >> Click on “Instances running” — It will show if any [EC2 instances](#) are running or not.

Step 2: Click on the launch instance click on the launch instance, after clicking on it you will be redirected to a launch page where we can create an instance. Configure all the requirements to Create a new instance like the name of the instance as shown in the figure below.

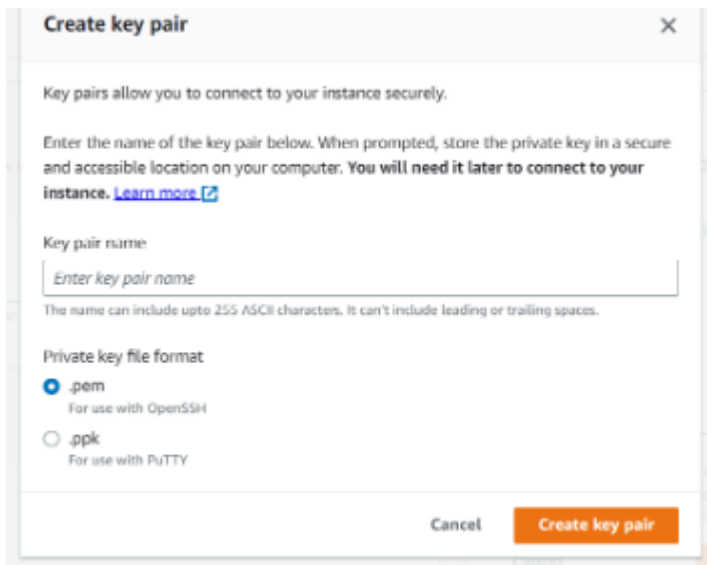


Step 3: Select AMI – Required operating system from the available. There are different types of OS available select the OS as per your requirement.



Selecting AMI

Step 4: By default, it selects a free tier of storage. (IF YOU ARE ELIGIBLE FOR THE FREE TIER). From the available storage specifications, select a free tier-eligible storage service. The instance type includes the number of CPUs required and the Memory required for your application. By default, the instance type is “t2.micro” which is a free tier-eligible service. Do not select any other which leads to the billing amount. To know more about instance types refer to [Amazon EC2 – Instance Types](#).



Step 5: Keep the network settings as default settings and make changes if required. Storage As mentioned in the picture, Free tier eligible can get up to 30 GB of [EBS Storage](#). Keep it as default.

▼ **Configure storage** Info Advanced

1x 30 GiB gp2 ▼ Root volume (Not encrypted)

i Free tier eligible customers can get up to 30 GB of EBS General Purpose (SSD) or Magnetic storage X

Add new volume

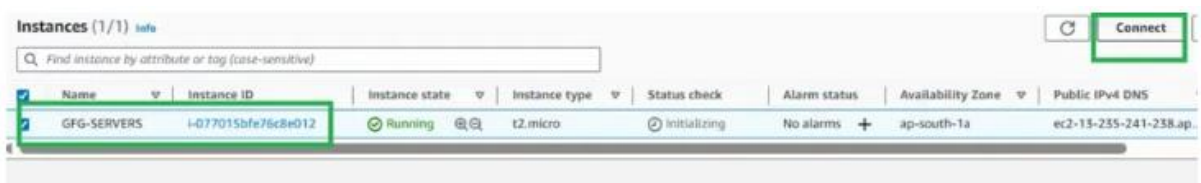
The selected AMI contains more instance store volumes than the instance allows. Only the first 0 instance store volumes from the AMI will be accessible from the instance

0 x File systems Edit

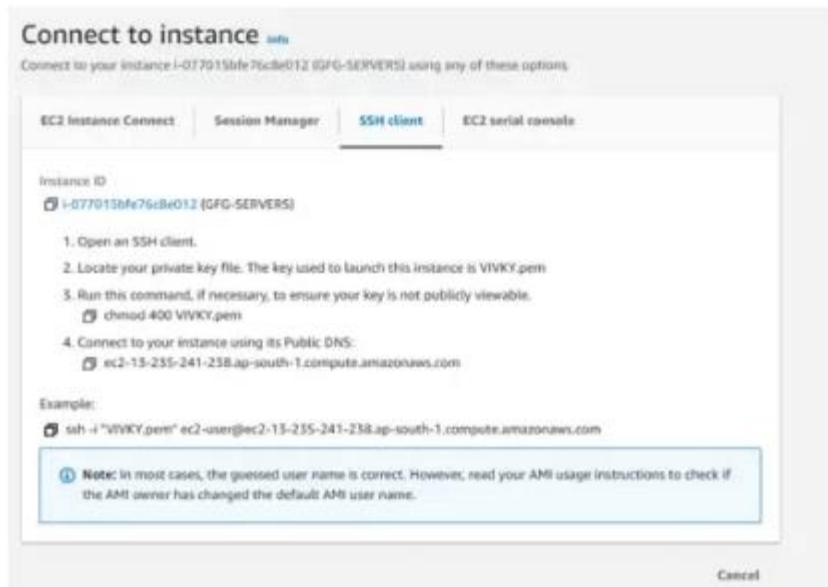
Step 6: Launching Instance At last, Check if all the selected are eligible for a free tier or not and click on “Launch instance”. That’s it, an instance will be created.

Steps To Connect Terminal Using SSH-Key:

Step 1: Select the server to which you want to connect and click on the connect button at the top of that instance as shown in the image below.



Step 2: Copy the SSH key which is right following the example it will act as a [key-pair](#) to connect to EC2-Instance.



Step 3: Open the terminal and go to the folder where your .pem file is located and paste the key that you have copied in AWS and paste it in the [terminal](#).

```
PS C:\Users\rknay\Downloads> ssh -i "VIVKY.pem" ec2-user@ec2-13-235-241-238.ap-south-1.compute.amazonaws.com
The authenticity of host 'ec2-13-235-241-238.ap-south-1.compute.amazonaws.com (13.235.241.238)' can't be established.
ED25519 key fingerprint is SHA256:SVxqQUp4UBe9rUMXvZ1uL9UnzRNfpSFk8DjMybXVoyE.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? YES
Warning: Permanently added 'ec2-13-235-241-238.ap-south-1.compute.amazonaws.com' (ED25519) to the list of known hosts.
Register this system with Red Hat Insights: insights-client --register
Create an account or view all your systems at https://red.ht/insights-dashboard
[ec2-user@ip-172-31-34-45 ~]$
```

4. Develop a simple applications using Apex programming language of salesforce.com

Aim: To develop a simple custom application using the Apex programming language on the Salesforce cloud platform.

PROCEDURE:

Step 1: Sign up for a Salesforce Developer Org: If you don't have one, sign up for a free developer account on the Salesforce Signup Page.

Step 2: Log in to your Developer Org: Access your Salesforce instance with your credentials.

Step 3: Open the Developer Console: Click the **gear icon** (Setup) in the top-right corner of the Salesforce Classic interface or the top-left in Lightning Experience, and then select **Developer Console**.

Step 4: Create a New Apex Class:

1. In the Developer Console, navigate to **File > New > Apex Class**.
2. Enter a name for your class (e.g., HelloWorldApp) and click **OK**.

Step 5: Write the Apex Code: A basic class structure will be automatically generated. Add a method within the curly braces to perform a simple action, such as printing a message to the debug log.

Apex Code

```
public class HelloWorldApp {  
    public static void sayHello() {  
        System.debug('WELCOME TO APEX PROGRAMMING');  
    }  
}
```

Ensure you click **File > Save** to save your new class.

Step 6: Execute the Apex Code:

1. In the Developer Console, click the **Debug** menu and select **Open Execute Anonymous Window**.
2. In the window that opens, enter the following code to call your method:

Apex Code

```
HelloWorldApp.sayHello();
```

3. Ensure the **Open Log** checkbox is selected and click the **Execute** button.

Step 7: View the Output: The execution log will open automatically. To see only your output, ensure the **Debug Only** checkbox is selected in the log inspector. Your message "WELCOME TO APEX PROGRAMMING" will be displayed in the log.

5. Implement a mailing service using Apex programming language of salesforce.com

Aim: Implement a mailing service using Apex programming language of salesforce.com

Procedure:

Step 1: Open the Developer Console To open the Developer Console, you can either click on “Your Name” or the quick access menu (Setup gear icon) on the top right corner of the Salesforce page.

Step 2: Create a new Apex class In the Developer Console, click on “File” and then “New” and select “Apex Class”. Enter “EmailManager” as the class name and then click “OK”.

Step 3: Replace the default class body Replace the default class body with the following example of the EmailManager class.

```
public class EmailManager {  
    // Public method  
    public void sendMail(String address, String subject, String body) {  
        // Create an email message object  
        Messaging.SingleEmailMessage mail = new Messaging.SingleEmailMessage();  
        String[] toAddresses = new String[] {address};  
        mail.setToAddresses(toAddresses);  
        mail.setSubject(subject);  
        mail.setPlainTextBody(body);  
        // Pass this email message to the built-in sendEmail method  
        // of the Messaging class  
        Messaging.SendEmailResult[] results = Messaging.sendEmail(  
                                                    new Messaging.SingleEmailMessage[] { mail });  
        // Call a helper method to inspect the returned results  
        inspectResults(results);  
    }  
}
```

```
private static Boolean inspectResults(Messaging.SendEmailResult[] results) {
    Boolean sendResult = true;
    // sendEmail returns an array of result objects.
    // Iterate through the list to inspect results.
    // In this class, the methods send only one email,
    // so we should have only one result.
    for (Messaging.SendEmailResult res : results) {
        if (res.isSuccess()) {
            System.debug('Email sent successfully');
        }
        else {
            sendResult = false;
            System.debug('The following errors occurred: ' + res.getErrors());
        }
    }
    return sendResult;
}
```

Step 4: Save the class Click on “File” and then “Save” to save your class. Once saved, you can use this class to send emails in other Apex classes or triggers.

Step 6: Test the output

1. In the Developer Console, click **Debug | Open Execute Anonymous Window**.
2. In the window that opens, enter the following. Replace ‘Your email addresses with your email address.

```
EmailManager em = new EmailManager();
em.sendMail('Your email address', 'Email Subject', 'Email Body');
```

Click Execute. Check your email!

If you want to call the method directly without using “new”, use the “static” keyword and call it as follows;

```
public static void sendMail(String ...
...
```

```
EmailManager.sendMail('Your email address', 'Email Subject', 'Email Body');
```

6. Simulate a cloud scenario using CloudSim and run a scheduling algorithm that is not present in CloudSim.

Aim: To Simulate a cloud scenario using CloudSim and run a scheduling algorithm that is not present in Cloud Sim.

PROCEDURE:

To use CloudSim in Eclipse:

Step 1: Download CloudSim installable files from

<https://code.google.com/p/cloudsim/downloads/list> and unzip

Step 2: Open Eclipse

Step 3: Create a new Java Project: File ->New

Step 4: Import an unpacked CloudSim project into the new Java Project

Step 5: The first step is to initialize the CloudSim package by initializing the CloudSim library as follows:

```
CloudSim.init(num_user, calendar, trace_flag)
```

Step 6: Data centres are the resource providers in CloudSim; hence, creation of data centres is a second step. To create Datacenter, you need the Data center Characteristics object that stores the properties of a data centre such as architecture, OS, list of machines, allocation policy that covers the time or space shared, the time zone and its price:

```
Datacenter data_center_9883 = new Datacenter(name, characteristics, new Vm Allocation Policy Simple(host List))
```

Step 7: The third step is to create a broker:

```
DatacenterBroker broker = createBroker();
```

Step 8: The fourth step is to create one virtual machine unique ID of the VM, userId ID of the VM's owner, mips, number of PEs amount of CPUs, amount of RAM, amount of bandwidth, amount of storage, virtual machine monitor, and cloud let Scheduler policy for cloudlets:

```
Vm vm = new Vm(vmid, brokerId, mips, pesNumber, ram, bw, size, vmm, new CloudletSchedulerTimeShared())
```

Step 9: Submit the VM list to the broker: `broker.submitVmList(vmlist)`

Step 10: Create a cloudlet with length, file size, output size, and utilisationmodel:

Step 12: Start the simulation: `CloudSim.startSimulation()`

CloudSimExample1...

=====OUTPUT=====

Cloudlet ID STATUS Data center ID VM ID Time Start Time Finish Time

0 SUCCESS 20 400 0.1 400.1

*****Datacenter: Datacenter_0***** Userid Debt

3 35.6

CloudSimExample1 finished!

7. Implementation of thread based image processing application in Microsoft Azure

Aim: To implement thread based image processing application in Microsoft Azure

PROCEDURE:

Step 1: Create Azure Account

1. Go to Microsoft Azure
2. Sign in or create a new account
3. Open **Azure Portal**

Step 2: Create Storage Account

1. In Azure Portal, search for **Storage Accounts**
2. Click **Create**
3. Fill details:
 - Resource Group: Create new
 - Storage Account Name: imagestorage123
 - Region: Choose nearest (e.g., Central India)
4. Click **Review + Create** → **Create**

Step 3: Create Blob Container

1. Open the created Storage Account
2. Go to **Containers**
3. Click + **Container**
4. Name: images
5. Access level: Private
6. Click **Create**

Step 4: Upload Sample Images

1. Open the images container
2. Click **Upload**
3. Select multiple image files
4. Click **Upload**

Step 5: Get Connection String

1. Go to **Access Keys** in Storage Account
2. Copy **Connection String**
3. Save it for use in code

Step 6: Install Required Libraries

Run the following in your system:

```
pip install azure-storage-blob pillow
```

Step 7: Write Multithreaded Python Code

```
import threading
from azure.storage.blob import BlobServiceClient
from PIL import Image
import io

# Azure connection
connection_string = "YOUR_CONNECTION_STRING"
container_name = "images"

blob_service_client = BlobServiceClient.from_connection_string(connection_string)

def process_image(blob_name):
    blob_client = blob_service_client.get_blob_client(container=container_name, blob=blob_name)

    # Download image
    data = blob_client.download_blob().readall()
    stream = io.BytesIO(data)

    # Open and process image
    img = Image.open(stream)
    img = img.resize((200, 200)) # Resize

    # Save processed image
    output = io.BytesIO()
    img.save(output, format='JPEG')
    output.seek(0)

    # Upload processed image
    new_name = "processed_" + blob_name
    blob_service_client.get_blob_client(container=container_name,
    blob=new_name).upload_blob(output, overwrite=True)

def main():
```

```
container_client = blob_service_client.get_container_client(container_name)
blobs = container_client.list_blobs()

threads = []

# Create threads
for blob in blobs:
    t = threading.Thread(target=process_image, args=(blob.name,))
    threads.append(t)
    t.start()

# Wait for completion
for t in threads:
    t.join()

print("All images processed successfully")

if __name__ == "__main__":
    main()
```

Step 8: Run the Application

1. Replace "YOUR_CONNECTION_STRING" with actual value
2. Run the script: **python app.py**

Step 9: Verify Output

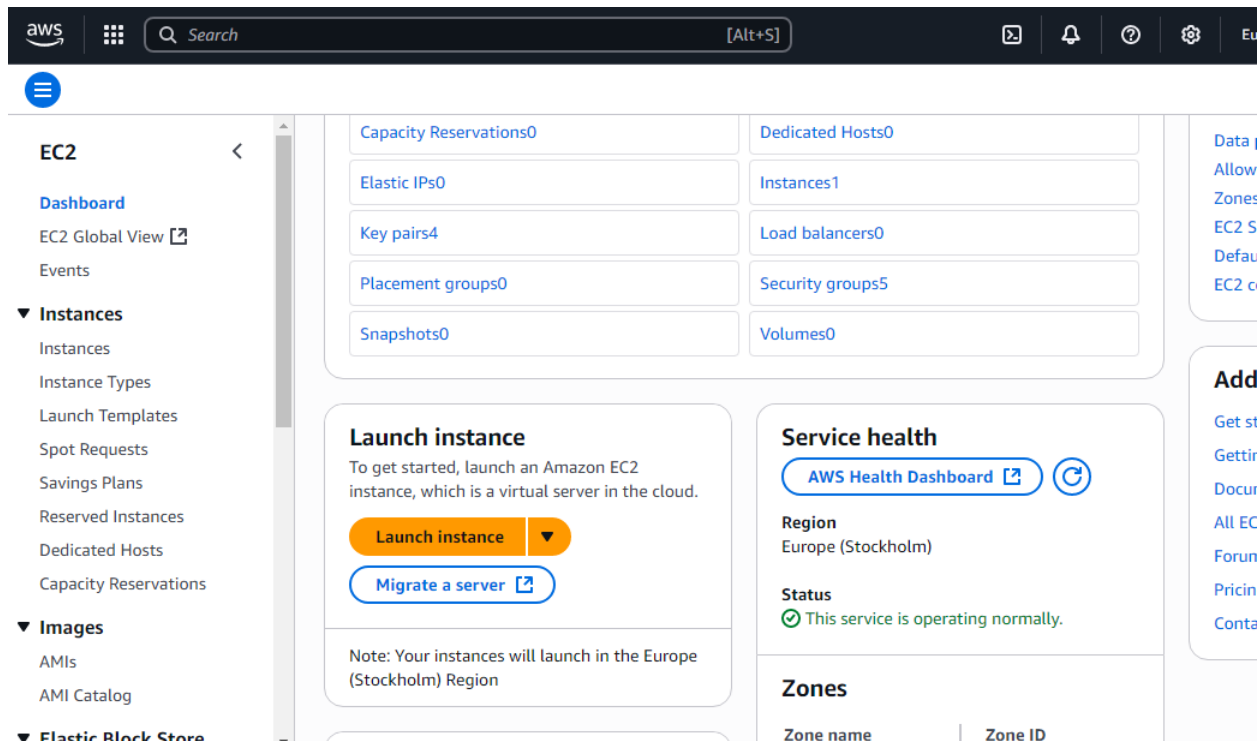
1. Go back to Azure Portal
2. Open **Blob Container (images)**
3. Check for new files:
 - processed_image1.jpg
 - processed_image2.jpg

8. Deploy dynamic web application on EC2 Instance on AWS

Aim: To deploy dynamic web application on EC2 Instance on AWS

PROCEDURE:

Step 1: Launch an instance



Type EC2 in search box, Click on EC2 , Later EC2 page will open as shown above Click on Launch instance

Step 2: Name and tags

Launch an instance Info

Amazon EC2 allows you to create virtual machines, or instances, that run on the AWS Cloud. Quickly get started by following the simple steps below.

Name and tags Info

Name

[Add additional tags](#)

Under Name and tags give any Name like web server

Step 3: Under Application and OS images select **Amazon Linux AWS**

▼ Application and OS Images (Amazon Machine Image) Info

An AMI contains the operating system, application server, and applications for your instance. If you don't see a suitable AMI below, use the search field or choose **Browse more AMIs**.

Q

Recents

Quick Start

Amazon Linux

aws

macOS

Mac

Ubuntu

ubuntu

Windows

Microsoft

Red Hat

Red Hat

SUSE Linux

SUSE

De

de

Q

Browse more AMIs

Including AMIs from AWS, Marketplace and the Community

Amazon Machine Image (AMI)

Amazon Linux 2023 kernel-6.1 AMI

ami-0c4fc5dcabc9df21d (64-bit (x86), uefi-preferred) / ami-07ca8bbd22d87429f (64-bit (Arm), uefi)

Virtualization: hvm ENA enabled: true Root device type: ebs

Step 4: Under Instant type select **t3.micro**

▼ Instance type Info | Get advice

Instance type

t3.micro

Free tier eligible

Family: t3 2 vCPU 1 GiB Memory Current generation: true

On-Demand Ubuntu Pro base pricing: 0.0143 USD per Hour

On-Demand RHEL base pricing: 0.0396 USD per Hour

On-Demand SUSE base pricing: 0.0108 USD per Hour

On-Demand Linux base pricing: 0.0108 USD per Hour

On-Demand Windows base pricing: 0.02 USD per Hour

▼

All generations

Compare instance types

Additional costs apply for AMIs with pre-installed software

Department of Computer Science & Engineering, SCEM, Mangaluru.

22

Step 5: Under Key Pair click on create new key Pair and give name to key Pair like kyp and click on create key pair, kyp.pm file will be downloaded.

The screenshot shows the 'Create key pair' dialog box. It has a title bar with a close button (X). The main content area is titled 'Create key pair'. Under 'Key pair name', there is a text input field containing 'kyp' and a note: 'The name can include up to 255 ASCII characters. It can't include leading or trailing spaces.' Under 'Key pair type', there are two radio button options: 'RSA' (selected) and 'ED25519'. Under 'Private key file format', there are two radio button options: '.pem' (selected) and '.ppk'. At the bottom, there is a warning box with a yellow triangle icon: 'When prompted, store the private key in a secure and accessible location on your computer. You will need it later to connect to your instance. Learn'. At the very bottom, there are two buttons: 'Cancel' and 'Create key pair'.

In network settings make sure allow SSH traffic from is enabled, and enable by clicking the check box of allow HTTPS traffic from internet and check box of allow HTTP traffic from internet

The screenshot shows the 'Network settings' page. It has a title bar with a dropdown arrow and the text 'Network settings' followed by an 'Info' link. On the right, there is an 'Edit' button. The main content area is titled 'Network settings' and includes sections for 'Network', 'Subnet', 'Auto-assign public IP', and 'Firewall (security groups)'. The 'Firewall (security groups)' section is expanded, showing a 'Create security group' button (selected) and a 'Select existing security group' button. Below this, it says 'We'll create a new security group called 'launch-wizard-5' with the following rules:'. There are three rules listed, each with a checked checkbox: 'Allow SSH traffic from' (with a dropdown menu showing 'Anywhere' and '0.0.0.0/0'), 'Allow HTTPS traffic from the internet', and 'Allow HTTP traffic from the internet'.

In Configure Storage keep as it is

▼ **Configure storage** [Info](#) Advanced

1x GiB ▼ Root volume, 3000 IOPS, Not encrypted

Free tier eligible customers can get up to 30 GB of EBS General Purpose (SSD) or Magnetic storage

[Add new volume](#)

In Advanced details keep as it is.

EC2 > [Instances](#) > Launch an instance

▼ **Configure storage** [Info](#) Advanced

1x GiB ▼ Root volume, 3000 IOPS, Not encrypted

Free tier eligible customers can get up to 30 GB of EBS General Purpose (SSD) or Magnetic storage

[Add new volume](#)

Click refresh to view backup information
The tags that you assign determine whether the instance will be backed up by any Data Lifecycle Manager policies.

0 x File systems
[Edit](#)

► **Advanced details** [Info](#)

Summary

Number of instances [Info](#)

Software Image (AMI)
Amazon Linux 2023 AMI 2023.8.2...[read more](#)
ami-0c4fc5dcabc9df21d

Virtual server type (instance type)
t3.micro

Firewall (security group)
New security group

Storage (volumes)
1 volume(s) - 8 GiB

[Cancel](#) [Launch instance](#) [Preview code](#)

Again click on Launch Instance

aws [Alt+S]

EC2

Dashboard
EC2 Global View [↗](#)
Events

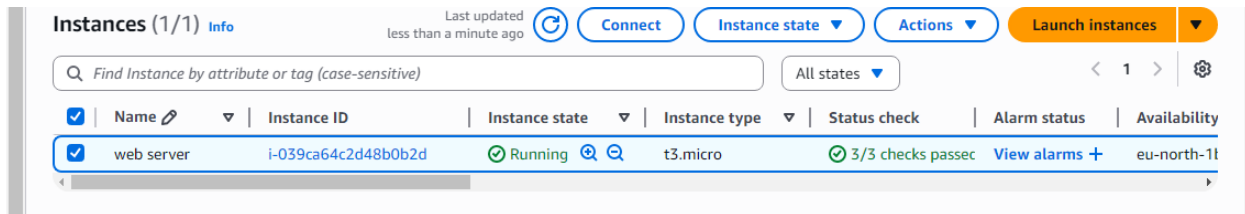
▼ **Instances**
Instances
Instance Types
Launch Templates

Resources [EC2 Global View](#) [⚙](#) [↺](#)

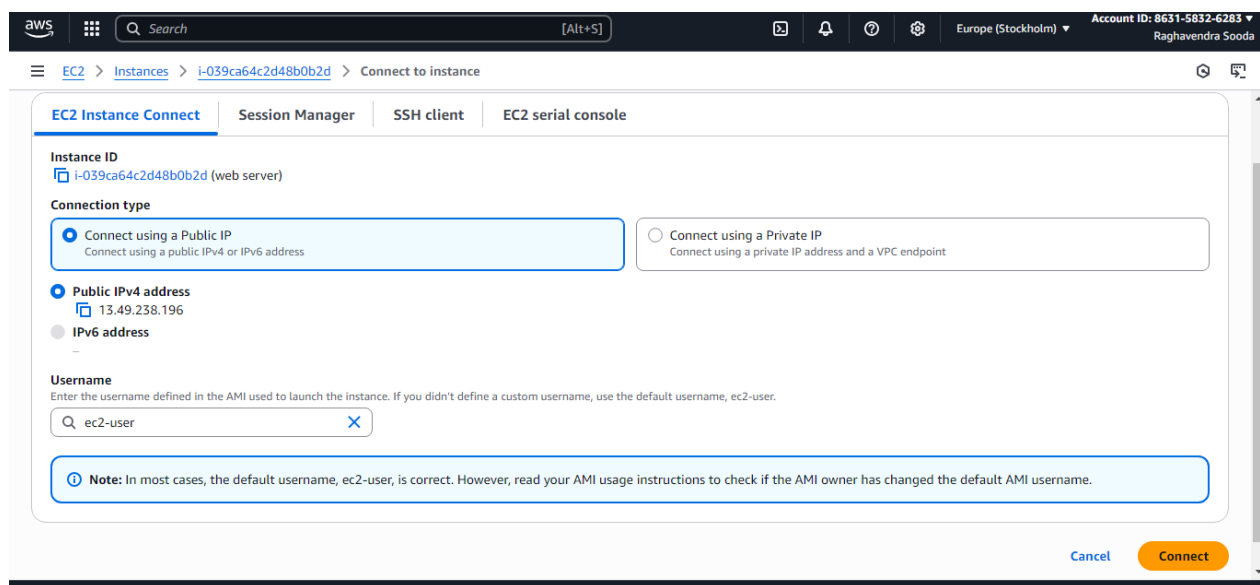
You are using the following Amazon EC2 resources in the Europe (Stockholm) Region:

Instances (running)0	Auto Scaling Groups0
Capacity Reservations0	Dedicated Hosts0
Elastic IPs0	Instances1
Key pairs4	Load balancers0

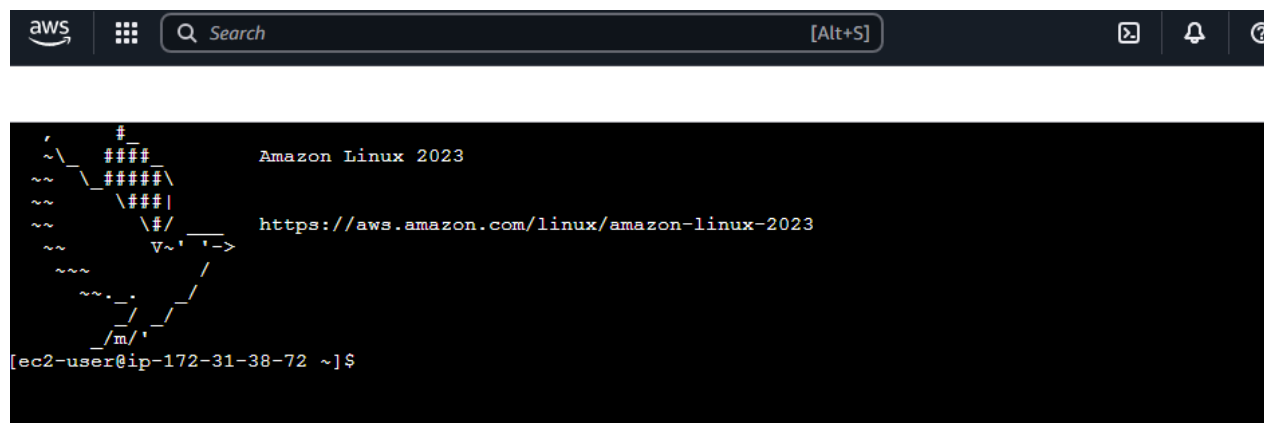
After Successful Launching click on Instances



Select the check box of newly selected instance named web- server click on connect button



Under Connect select EC2 Instance Connect method and then press connect button.

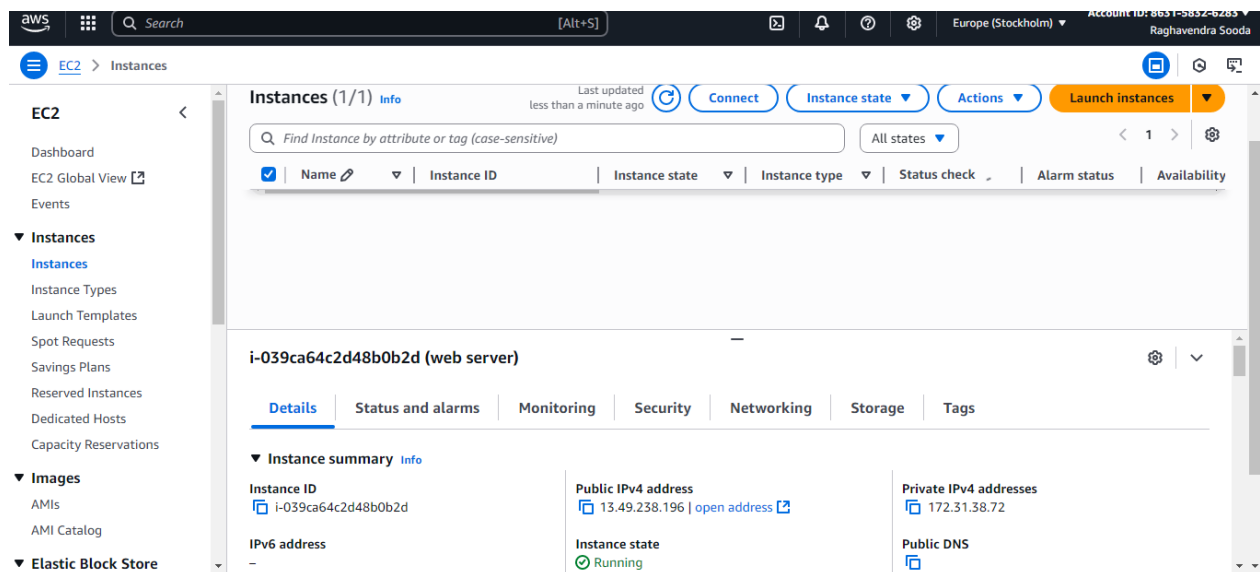


Type following steps

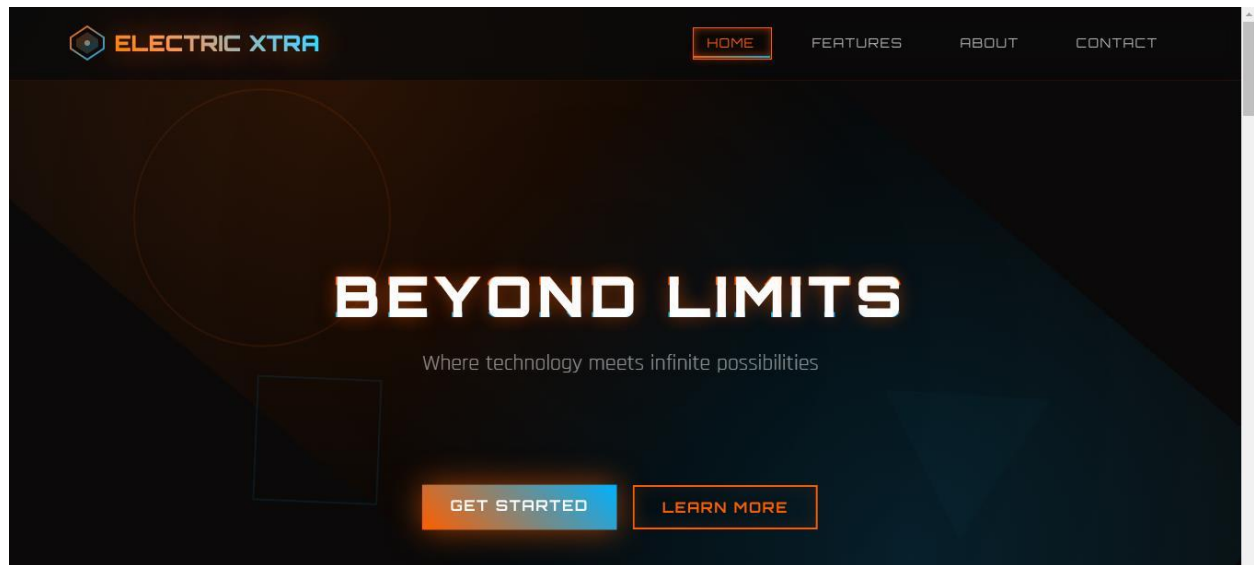
```

sudo su - ;root
user yum update -y
:update
yum install -y httpd ; install httpd server
mkdir temp
cd temp
wget https://templatemo.com/download/templatemo_596_electric_xtra ;download website
template files
ls -lrt
mkdir templatemo_596_electric_xtra_unzipped
unzip templatemo_596_electric_xtra -d templatemo_596_electric_xtra_unzipped
cd templatemo_596_electric_xtra_unzipped
ls -lrt
cd templatemo_596_electric_xtra
ls -lrt
mv * /var/www/html/ ; copy the extracte file
/var/www/html cd /var/www/html/
ls -lrt
systemctl status httpd ; to check the status of httpd
systemctl enable httpd ;enable the httpd server
systemctl start httpd ; start the httpd server
    
```

click on the instances



copy the public ipv4 address 13.49.238.1969 (in my case) go to browser window paste the copied public ip of the instance. Web page will open as shown below



9. Use Google App Engine Launcher to launch web applications.

Aim: To use Google App Engine Launcher to launch web applications.

PROCEDURE:

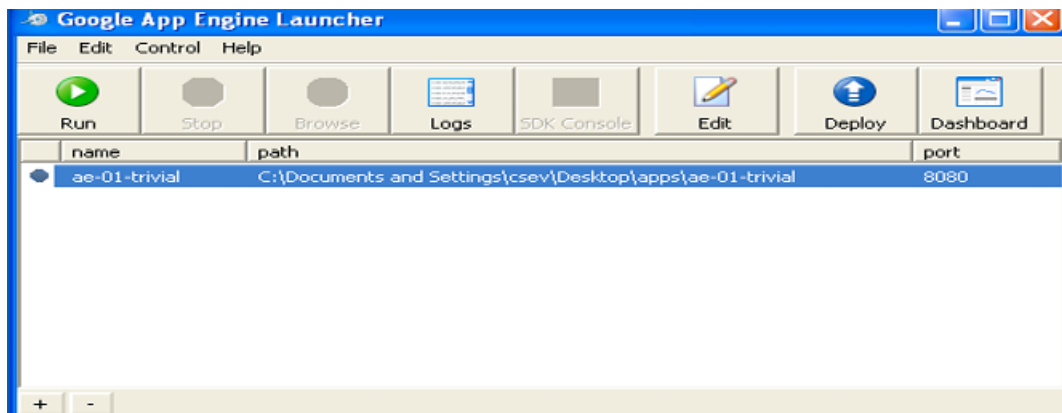
C:\Documents and Settings\csev\Desktop\apps And then make a sub---folder in within apps called “ae--01--trivial” – the path to this folder would be: C:\ Documents and Settings \csev\Desktop\apps\ae--01--trivial Using a text editor such as JEdit (www.jedit.org)

create a file called app.yaml in the ae--01--trivial folder with the following contents: application: ae-01-trivial version: 1 runtime: python api_version: 1 handlers:- url: /. * script: index.py

Then create a file in the ae--01--trivial folder called index.py with three lines in it: print 'Content-Type: text/plain' print '' print 'Hello there Chuck'

Then start the GoogleAppEngineLauncher program that can be found under Applications. Use the File --> Add Existing Application command and navigate into the apps directory and select the ae--01--trivial folder.

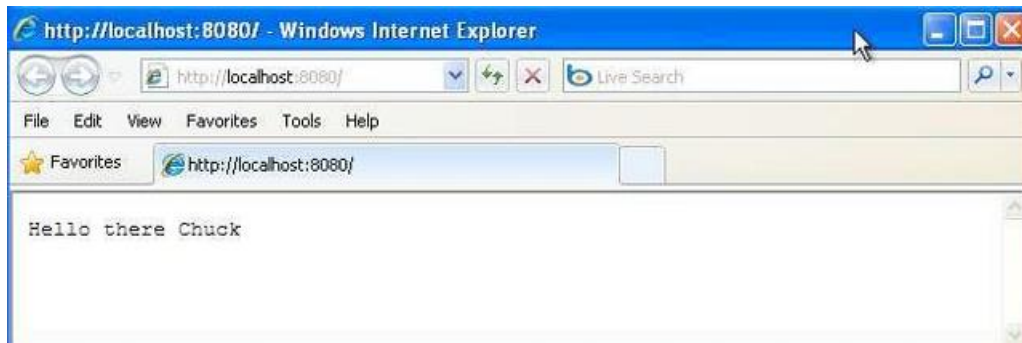
Once you have added the application, select it so that you can control the application using the launcher.



Once you have selected your application and press Run. After a few moments your application will start and the launcher will show a little green icon next to your application.

Then press Browse to open a browser pointing at your application which is running at <http://localhost:8080/>

Paste <http://localhost:8080> into your browser and you should see your application as follows:



Edit the index.py to change the name “Chuck” to your own name and press Refresh in the browser to verify your updates.

Watching the Log You can watch the internal log of the actions that the web server is performing when you are interacting with your application in the browser.

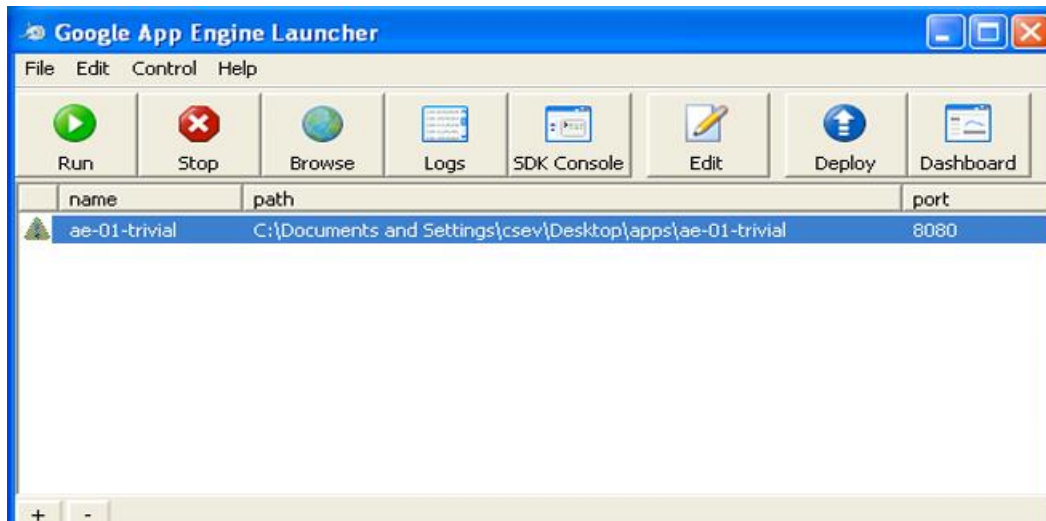
Select your application in the Launcher and press the Logs button to bring up a log window:

Each time you press Refresh in your browser – you can see it retrieving the output with a GET request.



Dealing with Errors

With two files to edit, there are two general categories of errors that you may encounter. If you make a mistake on the app.yaml file, the App Engine will not start, and your launcher will show a yellow icon near your application:



To get more detail on what is going wrong, take a look at the log for the application:



Result:

Thus the GAE web applications were created.

10. Deploy static web application using S3 on AWS and secure it with signed URL's

Aim: To deploy static web application using S3 on AWS and secure it with signed URL's

PROCEDURE:

Step 1: Create AWS S3 bucket:

- On AWS console move to S3 and create bucket,
- Select AWS region
- Enter the Bucket name
- Uncheck the "Block all public access"
- leave the rest of the settings default and press Create Bucket

Amazon S3 > Buckets > Create bucket

Create bucket [Info](#)

Buckets are containers for data stored in S3. [Learn more](#)

General configuration

AWS Region

US East (Ohio) us-east-2

Bucket name [Info](#)

deploy-app-on-s3

Bucket name must be unique within the global namespace and follow the bucket naming rules. [See rules for bucket naming](#)

Copy settings from existing bucket - optional

Only the bucket settings in the following configuration are copied.

[Choose bucket](#)

Format: s3://bucket/prefix

Object Ownership [Info](#)

Control ownership of objects written to this bucket from other AWS accounts and the use of access control lists (ACLs). Object ownership determines who can specify access to objects.

☒ **ACLs disabled (recommended)**

All objects in this bucket are owned by this account. Access to this bucket and its objects is specified using only policies.

☐ **ACLs enabled**

Objects in this bucket can be owned by other AWS accounts. Access to this bucket and its objects can be specified using ACLs.

Object Ownership

Bucket owner enforced

Step 2: Once it is created then select "Properties", scroll down to bottom, "Edit" the static website hosting and set the below settings and press "Save changes",

Amazon S3 > Buckets > deploy-app-on-s3 > Edit static website hosting

Edit static website hosting [Info](#)

Static website hosting
Use this bucket to host a website or redirect requests. [Learn more](#)

Static website hosting

☐ Disable

☒ Enable

Hosting type

☒ Host a static website
Use the bucket endpoint as the web address. [Learn more](#)

☐ Redirect requests for an object
Redirect requests to another bucket or domain. [Learn more](#)

For your customers to access content at the website endpoint, you must make all your content publicly readable. To do so, you can edit the S3 Block Public Access settings for the bucket. For more information, see [Using Amazon S3 Block Public Access](#)

Index document
Specify the home or default page of the website.

index.html

Error document - optional
This is returned when an error occurs.

index.html

Step 3: Then, select "Permissions", then "Edit" the Bucket policy, add the following content (remember to update the BUCKET_NAME) and Save Changes,

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "PublicReadGetObject",
      "Effect": "Allow",
      "Principal": "*",
      "Action": "s3:*",
      "Resource": [
        "arn:aws:s3:::BUCKET_NAME/*",
```

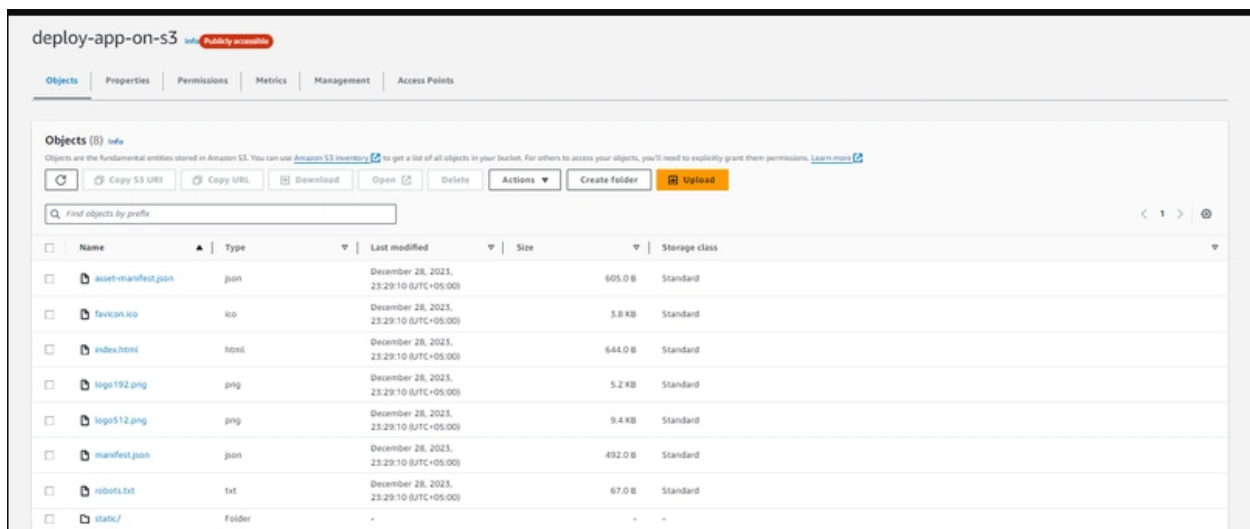
```

    "arn:aws:s3:::BUCKET_NAME"
  ]
}
]
}

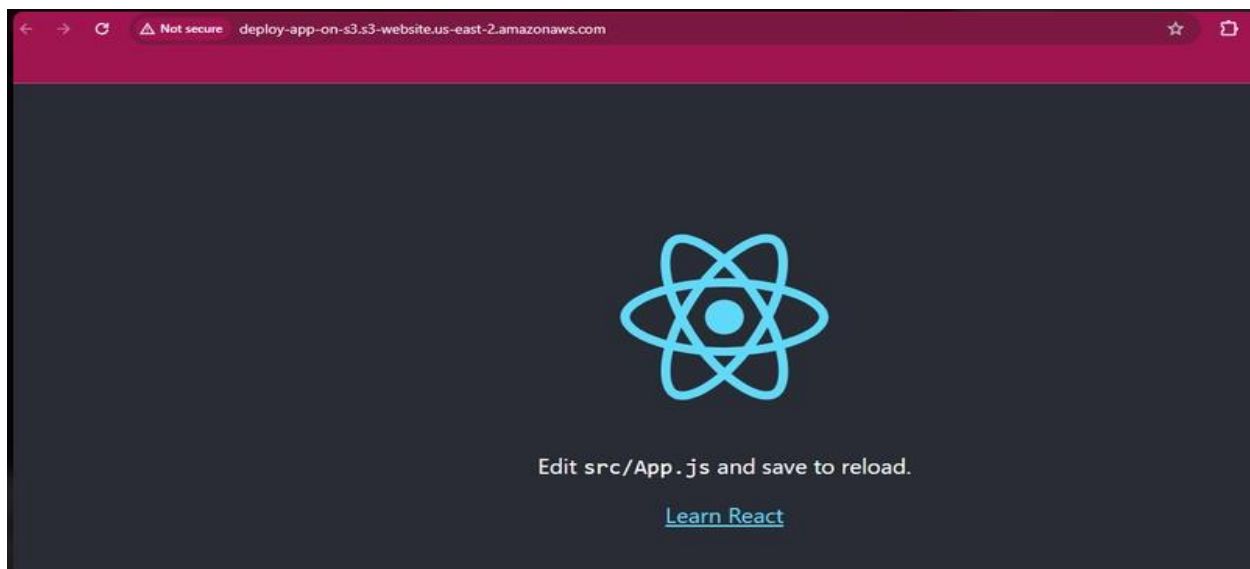
```

Step 4: Upload Your Files:

- Open your new bucket and click Upload.
- Drag and drop your website files (e.g., index.html, styles.css) and click Upload.



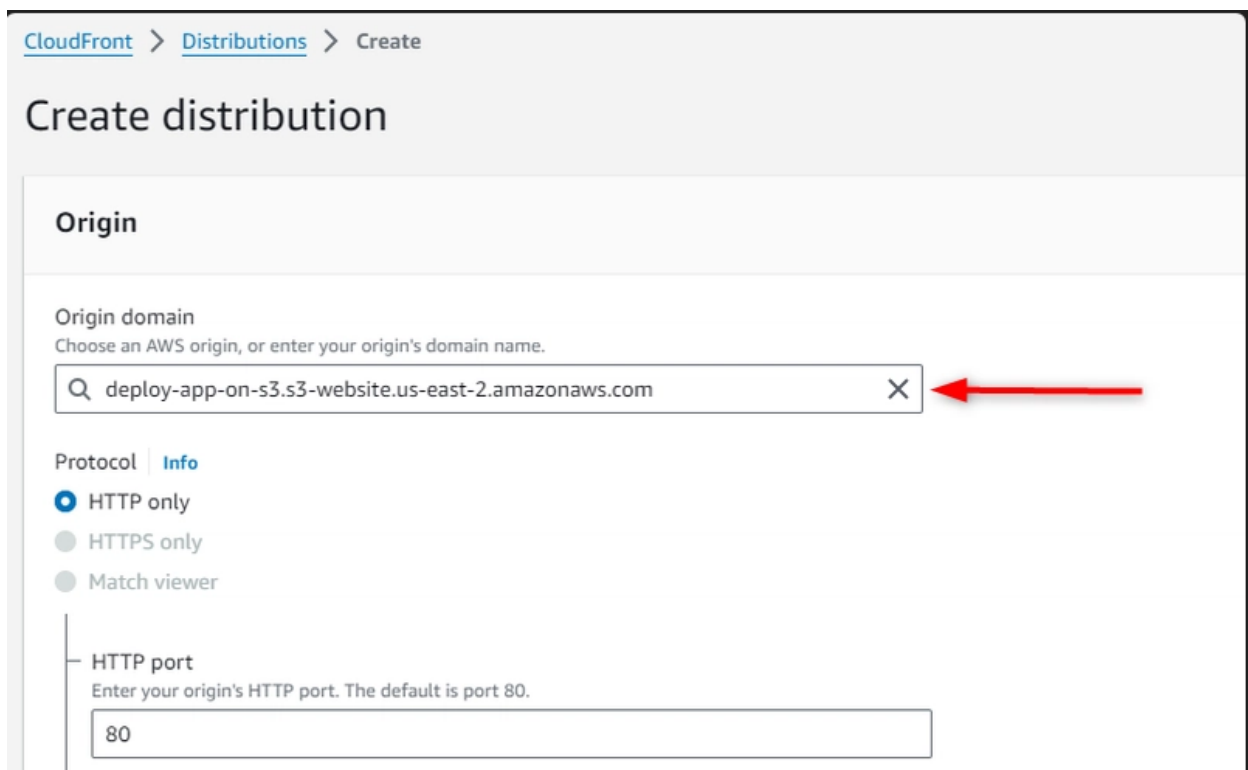
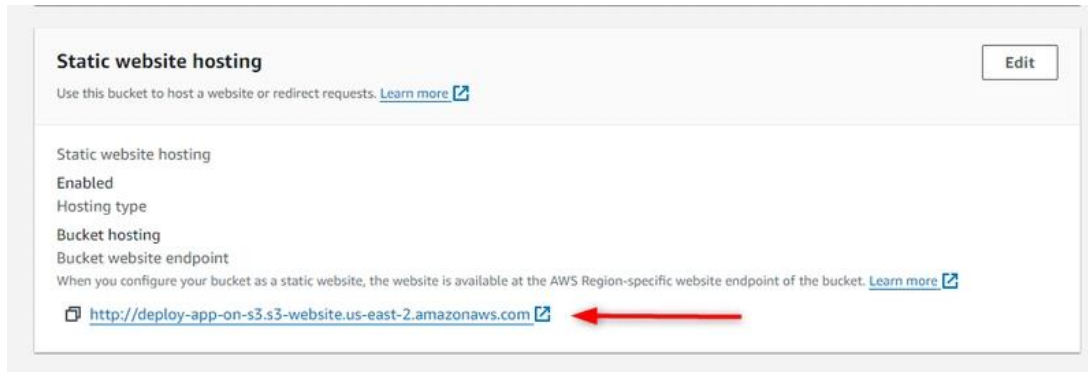
Now the static app will be accessible on the S3 static url,



Step 5: Attach Cloudfront with S3:

To access the static content from S3 with high speed and less latency we will attach the cloudfront with S3 bucket. To do so,

- move to Cloudfront from AWS console. Press the "Create Distribution"
- Copy the static url from the s3 bucket properties and place it in the origin domain for cloudfront. Don't forget to remove the https:// from the s3 bucket static url. Image description



- Then, select the Redirect HTTP to HTTPS and set the Allowed HTTP methods:

Viewer

Viewer protocol policy

☐ HTTP and HTTPS

☒ Redirect HTTP to HTTPS

☐ HTTPS only

Allowed HTTP methods

☐ GET, HEAD

☐ GET, HEAD, OPTIONS

☒ GET, HEAD, OPTIONS, PUT, POST, PATCH, DELETE

Cache HTTP methods

GET and HEAD methods are cached by default.

☒ OPTIONS

Restrict viewer access

If you restrict viewer access, viewers must use CloudFront signed URLs or signed cookies to access your content.

☒ No

☐ Yes

- Then press "Create Distribution"
- Once the distribution is created, you will be able to access your website on Distribution domain name:

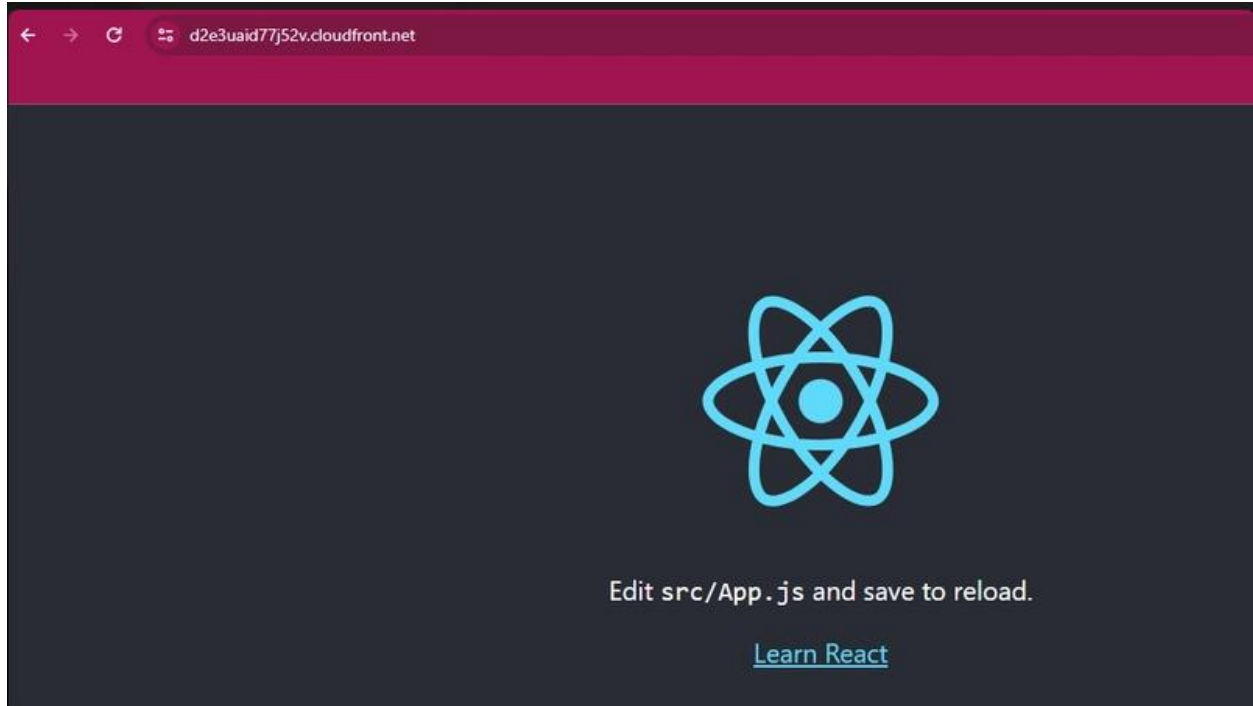
CloudFront > Distributions > E29HVFXACX6YMV

E29HVFXACX6YMV

General | Security | Origins | Behaviors | Error pages | Invalidations | Tags

Details

Distribution domain name	ARN
 d2e3uaid77j52v.cloudfront.net	 arn:aws:cloudfront::260923197968:distribution/E29HVFXACX6YMV



If you created your static website only as a learning exercise, delete the AWS resources that you allocated so that you no longer accrue charges. After you delete your AWS resources, your website is no longer available.

11. Create Video streaming service using S3 and cloudfront and AWS Elemental Media Convert Digital Rights Management System

Step 1: Set Up Your Amazon S3 Bucket

Storing raw video assets in a secure S3 bucket prevents unauthorized direct downloads while remaining available for your CDN.

1. Login to AWS Management Console. Navigate to: Services → S3
2. Click **Create bucket**.
 - Enter a globally unique name (e.g., my-video-streaming-bucket-2026).
 - Select your preferred AWS Region (e.g., us-east-1 or your closest localized region).
3. **Configure Settings**:
 - **Block Public Access**: Ensure **Block all public access** remains checked. Your bucket must remain private to force users through CloudFront.
 - **Bucket Versioning**: Select **Enable**.
 - **Default Encryption**: Select **Enable** with SSE-S3.
4. Click the **Create bucket** button at the bottom.
5. **Upload a Video**: Click on your new bucket name, select **Upload**, add a test .mp4 video, and finish by clicking **Upload**.

Step 2: Create a CloudFront Distribution

CloudFront acts as the CDN, which retrieves your private S3 objects securely and distributes them to your users.

1. Search for and open the **CloudFront** console.
2. Click **Create distribution**.
3. **Origin Settings**:
 - **Origin domain**: Select the S3 bucket you created in Step 1.
 - **Origin access**: Select **Origin access control settings (recommended)**.
 - **Origin access control (OAC)**: Click **Create control setting**, leave the default settings, and hit **Create**. (*This securely connects CloudFront to your private S3 bucket*).
4. **Default Cache Behavior Settings**:
 - **Viewer protocol policy**: Select **Redirect HTTP to HTTPS**.
 - **Allowed HTTP methods**: Select GET, HEAD.
 - **Cache key and origin requests**: Choose **Cache policy and origin request policy (recommended)** -> Cache policy: CachingOptimized.
5. **Web Application Firewall (Optional)**: Select **Do not enable security protections** (for basic lab purposes).
6. **Create Distribution**: Click **Create distribution** at the bottom of the page.

Step 3: Update S3 Bucket Policy for CloudFront

Because your S3 bucket is private, you need to grant CloudFront permission to access the video files.

1. Once your CloudFront distribution is created, a banner will appear at the top prompting you to **Copy policy**. Copy the policy JSON provided.
2. Go back to your **S3 console** and click on your bucket.
3. Select the **Permissions** tab.
4. Scroll down to **Bucket policy** and click **Edit**.
5. Paste the copied CloudFront policy into the editor and click **Save changes**.

Example:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowCloudFrontServicePrincipal",
      "Effect": "Allow",
      "Principal": {
        "Service": "cloudfront.amazonaws.com"
      },
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::video-streaming-bucket/*"
    }
  ]
}
```

Step 4: Test Your Streaming Service

1. Go back to your **CloudFront distributions** page and wait for the status to change from *Deploying* to *Last modified* (with a green checkmark).
2. Copy the **Distribution domain name** (e.g., d111111abcdef8.cloudfront.net).
3. Open a new web browser tab, paste the domain name, and append the name of the uploaded video file to the end of the URL (e.g., https://cloudfront.net).
4. The video should load immediately in the browser via the CloudFront stream.